



SenaiStock Mobile

Controle de Estoque de Livros Didáticos — versão mobile

Aplicativo Android desenvolvido em Flutter que reproduz as funcionalidades, os perfis de acesso e as regras de negócio do sistema web SenaiStock, com banco de dados SQLite local.

Instituição	SENAI Limeira/SP
Equipe	Diogo Scherrer, Gabriel Furtunato, Gabriel Xavier
Ano	2026
Stack	Flutter 3.47 · Dart 3.13 · SQLite (sqflite) · Provider · Material Design 3
Plataforma	Android 7.0+ (API 24)
Projeto de referência	Projeto/ — Laravel 13 + Livewire 4

Índice

1. Visão Geral do Sistema
2. Stack Tecnológica
3. Estrutura do Projeto
4. Banco de Dados
5. Autenticação e Perfis de Acesso
6. Módulos do Sistema
7. Regras de Negócio
8. Camada de Dados (no lugar da API RESTful)
9. Serviço de Estoque
10. Notificações
11. Credenciais de Demonstração
12. Como Executar
13. Testes e Qualidade
14. Diferenças em Relação ao Projeto Web
15. Limitações Conhecidas e Próximos Passos
16. Log de Desenvolvimento

1. Visão Geral do Sistema

1.1 O que é

O **SenaiStock Mobile** é um aplicativo Android para controle de estoque de livros didáticos do SENAI. É a versão para celular do sistema SenaiStock: tem as mesmas funcionalidades, os mesmos perfis de usuário, as mesmas regras de negócio e os mesmos fluxos de tela do sistema web desenvolvido em Laravel ([Projeto/](#)), adaptados para a interação por toque.

Ele foi construído **do zero e de forma independente**: não altera o projeto original e não depende do servidor Laravel para funcionar. Todo o sistema — usuários, livros, movimentações, reservas e notificações — roda dentro do próprio aparelho, com um banco de dados SQLite local.

1.2 O problema que resolve

O SENAI envia periodicamente grandes remessas de livros às unidades de ensino. Os livros ficam no almoxarifado e são retirados pelos instrutores para as turmas. O almoxarifado sabia quantos livros chegavam, mas perdia o controle de quantos saíam. Resultado: o estoque de um título importante zerava sem aviso, e a falta só era descoberta quando a turma precisava do material.

O SenaiStock resolve isso registrando **toda entrada e toda saída**, mantendo o saldo de cada título sempre atualizado e avisando quando um livro está abaixo do estoque mínimo.

1.3 Para que serve a versão mobile

Quem	Uso no dia a dia
Almoxarife	Registra entradas e saídas com o celular na mão, dentro do almoxarifado, no momento em que os livros chegam ou saem — sem precisar voltar ao computador. Acompanha alertas de baixo estoque e atende às reservas dos professores.
Professor	Consulta o saldo dos livros da sua matéria e faz a solicitação (reserva) dos livros para a turma direto pelo celular. Recebe aviso quando chegam livros da sua matéria.

1.4 O que o sistema oferece

- Cadastro de livros com ISBN único por título
- Registro de entradas (abastecimento) e saídas (retiradas para turmas)
- Saldo atualizado na hora, após cada movimentação
- Alerta de livros com estoque abaixo do mínimo configurado
- Histórico completo de movimentações, com filtros por tipo, livro e período
- Sistema de reservas: o professor solicita os livros e o almoxarife dá baixa
- Notificação ao professor quando chegam livros da sua matéria
- Controle de acesso por perfil (Almoxarife / Professor)
- Gerenciamento completo de usuários pelo almoxarife
- Sessão persistente: o usuário continua logado ao fechar e reabrir o app

1.5 Perfis de usuário

Perfil (tela)	Valor interno	Papel
Almoxarife	almoxarife	Opera o estoque: livros, entradas, saídas, baixa de reservas e usuários.
Professor	coordenador	Consulta livros da sua matéria, faz reservas e recebe notificações.

O valor interno `coordenador` foi mantido igual ao do projeto web (enum `PerfilUsuario`) para que os dois sistemas usem o mesmo vocabulário; na interface ele sempre aparece como **Professor**.

2. Stack Tecnológica

2.1 Linguagem, framework e interface

Tecnologia	Versão	Função no projeto
Dart	3.13.4	Linguagem de todo o código do app (telas, regras de negócio, banco).
Flutter	3.47.5 (canal stable)	Framework de interface multiplataforma do Google. Desenha cada tela com seu próprio motor gráfico e gera o app Android nativo.
Material Design 3	(incluso no Flutter)	Sistema de design dos componentes: cards, chips, barra de navegação inferior, diálogos, formulários.
Impeller (OpenGL ES)	(incluso no Flutter)	Motor de renderização usado pelo Flutter no Android.

2.2 Pacotes (dependências do `pubspec.yaml`)

Pacote	Versão	Para que é usado
<code>sqlite</code>	2.4.4	Banco de dados SQLite local: tabelas, consultas, transações atômicas. É o substituto do MySQL + Eloquent do projeto web.
<code>path</code>	1.9.1	Monta o caminho do arquivo do banco (<code>senaistock_mobile.db</code>) dentro da pasta de dados do app.
<code>shared_preferences</code>	2.5.5	Guarda o id do usuário logado para manter a sessão entre aberturas do app (equivalente à sessão web).
<code>provider</code>	6.1.5+1	Gerenciamento de estado: disponibiliza a sessão (<code>Session</code>) para todas as telas e reconstrói a interface no login/logout.
<code>crypto</code>	3.0.7	Gera o hash SHA-256 das senhas (equivalente ao <code>Hash::make()</code> do Laravel).
<code>uuid</code>	4.6.0	Gera o identificador único (UUID v4) de cada notificação, como na tabela <code>notifications</code> do Laravel.
<code>intl</code>	0.19.0	Formatação de datas no padrão brasileiro (<code>dd/MM/yyyy HH:mm</code>).
<code>cupertino_icons</code>	1.0.9	Conjunto de ícones padrão do template Flutter.

Dependências de desenvolvimento:

Pacote	Versão	Para que é usado
<code>flutter_test</code>	(SDK)	Framework de testes automatizados.
<code>flutter_lints</code>	4.0.0	Regras de análise estática (boas práticas e padrão de código Dart).

2.3 Plataforma Android e ferramentas de build

Ferramenta	Versão	Função
Android SDK — plataforma	API 36 (Android 16)	<code>compileSdk</code> e <code>targetSdk</code> do app.
Versão mínima suportada	API 24 (Android 7.0)	<code>minSdk</code> — padrão do Flutter 3.47.
Android SDK Build-Tools	36.0.0	Empacotamento do APK.
Android NDK	28.2.13676358 (r28c)	Ferramentas nativas exigidas pelo plugin Gradle do Flutter.
Android SDK Platform-Tools	37.0.1	<code>adb</code> : instala e depura o app no emulador/celular.
Gradle	9.3.1	Sistema de build do Android.
Android Gradle Plugin (AGP)	9.1.0	Integra o build Android ao Gradle.
Kotlin (plugin Gradle)	2.4.0	Compila a <code>MainActivity</code> (ponto de entrada Android do app).
JDK	Eclipse Temurin 21	Executa o Gradle; o código Android é compilado com alvo Java 17.
Android CLI (<code>cmdline-tools</code>)	1.0	Instala os pacotes do SDK (<code>android sdk install</code>) e cria/inicia emuladores (<code>android emulator</code>).
Android Emulator	37.2.12	Executa o app no computador.
Imagem do emulador	Android 16 (API 36), Google Play, x86_64	Dispositivo virtual <code>medium_phone</code> .

2.4 Ferramentas de desenvolvimento

Ferramenta	Uso
Visual Studio Code	Editor de código.
Git + GitHub	Versionamento.
Claude Code (assistente de IA da Anthropic)	Apoio na geração do código, na configuração do ambiente Android/Flutter e nesta documentação.
<code>flutter analyze</code> / <code>flutter test</code>	Análise estática e testes automatizados.

2.5 Por que Flutter?

- **Um único código-fonte** gera o app Android (e, no futuro, iOS) sem reescrever as telas.
- **Interface declarativa**, no mesmo espírito dos componentes Livewire do projeto web: cada tela é um componente com seu próprio estado, que se redesenha quando os dados mudam.
- **Hot reload**: alterações no código aparecem no emulador em menos de um segundo, sem reinstalar o app.

- **Material Design 3** já incluso, o que permite reproduzir a identidade visual do sistema web (azul, âmbar, verde e vermelho do Tailwind) com pouco código.

2.6 Por que um app independente com SQLite em vez de consumir a API Laravel?

O caminho natural seria o app mobile consumir a API RESTful do projeto web. Porém:

1. A API do projeto original cobre apenas **Livros, Estoque e Movimentações**. Reservas, Notificações, Usuários e Dashboard existem só nos componentes Livewire, sem endpoints.
2. Para o app ter as mesmas funcionalidades via API, seria necessário **alterar o projeto original** (criar controllers e rotas novos), e a decisão da equipe foi **não modificar o projeto de referência**.

Por isso o app tem sua própria camada de dados: as tabelas, as regras de negócio e os dados de demonstração do Laravel foram reimplementados em Dart sobre SQLite. A consequência está descrita em [Limitações](#): os dados ficam em cada aparelho.

2.7 Por que Provider?

O único estado realmente global do app é **quem está logado**. O `provider` resolve isso com uma única classe (`Session`), sem a complexidade de soluções maiores (Bloc, Riverpod). O restante do estado é local de cada tela, que consulta o banco ao abrir — o mesmo modelo dos componentes Livewire.

2.8 Comparativo de stack: web × mobile

Camada	Projeto web (Projeto/)	Projeto mobile (Projeto Mobile/)
Linguagem	PHP 8.3	Dart 3.13
Framework	Laravel 13	Flutter 3.47
Interface	Livewire 4 + Blade + Tailwind CSS 4 + Alpine.js	Widgets Flutter + Material Design 3
Banco de dados	MySQL 8.4 (servidor)	SQLite (arquivo local no aparelho)
Acesso a dados	Eloquent ORM	<code>sqflite</code> com SQL direto nos repositórios
Regras de negócio	<code>EstoqueService</code> + componentes Livewire	Repositórios (<code>lib/repositories/</code>)
Autenticação	Sessão web + Sanctum (API)	Sessão local (<code>shared_preferences</code>)
Hash de senha	bcrypt	SHA-256 (demonstração)
Controle de acesso	Middleware <code>role:almoxarife</code> / <code>role:coordenador</code>	Abas e botões montados conforme o perfil
Notificações	Canal <code>database</code> do Laravel	Tabela <code>notificacoes</code> local
Build	Composer + Vite	Gradle + Android SDK

3. Estrutura do Projeto

Projeto Mobile/	
├─ lib/	← Todo o código do app (Dart)
│ └─ main.dart	← Ponto de entrada: tema, Provider e decisão Login x Home
│ └─ core/	
│ └─ app_colors.dart	← Paleta (mesmas cores do Tailwind do projeto web)
│ └─ app_theme.dart	← Tema Material 3 (AppBar, cards, campos, botões)
│ └─ app_exceptions.dart	← Exceções de domínio (estoque, quantidade, regra de negócio)
│ └─ password_hasher.dart	← Hash SHA-256 das senhas
│ └─ models/	← Entidades e enums
│ └─ perfil_usuario.dart	← Enum Almoхарife / Coordenador (Professor)
│ └─ tipo_movimentacao.dart	← Enum Entrada / Saída
│ └─ status_reserva.dart	← Enum Pendente / Retirada / Cancelada
│ └─ usuario.dart	
│ └─ livro.dart	← + estaBaixoEstoque, temSaldoSuficiente
│ └─ movimentacao.dart	
│ └─ reserva.dart	← + isPendente, temEstoqueSuficiente
│ └─ notificacao.dart	
│ └─ data/	
│ └─ app_database.dart	← Schema SQLite (= migrations) + dados de demonstração (= seeders)
│ └─ repositories/	← Regras de negócio e acesso ao banco
│ └─ auth_repository.dart	
│ └─ livro_repository.dart	
│ └─ estoque_repository.dart	← Equivale ao EstoqueService.php
│ └─ movimentacao_repository.dart	
│ └─ reserva_repository.dart	
│ └─ notificacao_repository.dart	
│ └─ usuario_repository.dart	
│ └─ dashboard_repository.dart	
│ └─ state/	
│ └─ session.dart	← Usuário logado (Provider + shared_preferences)
│ └─ widgets/	← Componentes reutilizáveis
│ └─ badges.dart	← Badges de saldo, tipo, status e perfil
│ └─ stat_card.dart	← Cards de estatística do Dashboard
│ └─ empty_state.dart	← Mensagem de lista vazia
│ └─ confirm_dialog.dart	← Diálogo de confirmação + mensagens (snackbar)
│ └─ screens/	← Telas (equivalem a app/Livewire/*)
│ └─ login_screen.dart	
│ └─ home_shell.dart	← Navegação inferior por perfil (equivale ao layout com sidebar)
│ └─ dashboard/dashboard_screen.dart	
│ └─ livros/	← livros_screen, livro_form_screen
│ └─ movimentacoes/	← movimentacoes_screen, registrar_movimentacao_screen
│ └─ reservas/	← reservas_screen, nova_reserva_screen
│ └─ notificacoes/	← notificacoes_screen
│ └─ usuarios/	← usuarios_screen, usuario_form_screen
├─ test/	
│ └─ widget_test.dart	← Testes das regras de negócio
├─ android/	← Projeto Android gerado pelo Flutter (Gradle, Manifest, MainActivity)
├─ pubspec.yaml	← Dependências e metadados do app
├─ analysis_options.yaml	← Regras de lint
├─ README.md	← Guia rápido
├─ DOCUMENTACAO.md	← Esta documentação (fonte em Markdown)
└─ DOCUMENTACAO.pdf	← Esta documentação em PDF

Tamanho do código

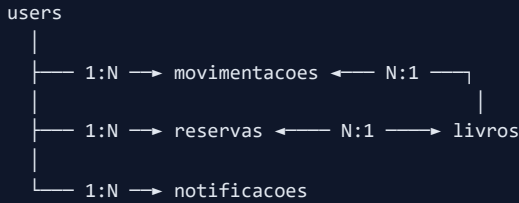
Pasta	Arquivos	Linhas
core/	4	117
models/	8	267

Pasta	Arquivos	Linhas
data/	1	182
repositories/	8	766
state/	1	53
widgets/	4	209
screens/	12	2.252
main.dart	1	44
Total lib/	39	3.890

4. Banco de Dados

O banco é um arquivo **SQLite** (`senaistock_mobile.db`) criado na pasta de dados do app na primeira abertura. O schema reproduz as migrations do projeto web.

4.1 Diagrama de Relacionamentos



As chaves estrangeiras são verificadas pelo SQLite (`PRAGMA foreign_keys = ON`).

4.2 Tabela `users`

Campo	Tipo	Descrição
id	INTEGER PK AUTOINCREMENT	Identificador único
nome	TEXT	Nome completo
email	TEXT UNIQUE	E-mail de acesso
senha_hash	TEXT	Hash SHA-256 da senha
perfil	TEXT	<code>almoxarife</code> ou <code>coordenador</code>
materia	TEXT NULL	Matéria do professor (só para <code>coordenador</code>)

Campo `materia` : determina quais livros o professor vê e de quais livros ele recebe notificações de chegada.

4.3 Tabela `livros`

Campo	Tipo	Descrição
id	INTEGER PK AUTOINCREMENT	Identificador único
titulo	TEXT	Título do livro
isbn	TEXT UNIQUE	ISBN — único (RN3)
materia	TEXT	Disciplina/matéria
saldo_atual	INTEGER DEFAULT 0	Quantidade em estoque (só muda por movimentações)
estoque_minimo	INTEGER DEFAULT 10	Limite para alerta de baixo estoque (RN6)

4.4 Tabela `movimentacoes`

Campo	Tipo	Descrição
id	INTEGER PK AUTOINCREMENT	Identificador único
livro_id	INTEGER FK → livros.id	Livro movimentado
user_id	INTEGER FK → users.id	Quem registrou (RN4)
tipo	TEXT	<code>entrada</code> ou <code>saida</code>
quantidade	INTEGER	Quantidade (sempre > 0, RN2)
observacao	TEXT NULL	Justificativa (obrigatória em saídas)
data_hora	TEXT (ISO 8601)	Data e hora exatas (RN4)

Imutável: movimentações nunca são editadas ou excluídas — formam o histórico de auditoria.

4.5 Tabela `reservas`

Campo	Tipo	Descrição
id	INTEGER PK AUTOINCREMENT	Identificador único
livro_id	INTEGER FK → livros.id	Livro reservado
user_id	INTEGER FK → users.id	Professor que fez a reserva
quantidade	INTEGER	Quantidade solicitada
status	TEXT	<code>pendente</code> , <code>retirada</code> ou <code>cancelada</code>
observacao	TEXT NULL	Turma / motivo
data_reserva	TEXT (ISO 8601)	Quando foi criada
data_retirada	TEXT NULL	Quando o almoxarife deu baixa

4.6 Tabela `notificacoes`

Campo	Tipo	Descrição
id	TEXT PK (UUID v4)	Identificador único
user_id	INTEGER FK → users.id	Professor que recebe
livro_id	INTEGER	Livro que chegou
livro_titulo	TEXT	Título do livro
materia	TEXT	Matéria do livro
quantidade	INTEGER	Quantidade que chegou
mensagem	TEXT	Texto exibido
read_at	TEXT NULL	Quando foi lida

Campo	Tipo	Descrição
created_at	TEXT (ISO 8601)	Quando foi criada

No Laravel, o conteúdo da notificação fica num campo JSON (`data`). Aqui cada informação tem sua própria coluna.

4.7 Diferenças de modelagem em relação ao MySQL

Aspecto	MySQL (web)	SQLite (mobile)
Campos ENUM	<code>ENUM('almoxarife', ...)</code>	<code>TEXT</code> , validado pelos enums do Dart
Datas	<code>TIMESTAMP</code>	<code>TEXT</code> no formato ISO 8601
<code>created_at</code> / <code>updated_at</code>	Automáticos em todas as tabelas	Não usados (exceto <code>notificacoes.created_at</code>)
Nomes de colunas	<code>name</code> , <code>password</code>	<code>nome</code> , <code>senha_hash</code>

4.8 Dados de demonstração

Criados automaticamente na primeira abertura (`AppDatabase._seed`), com o mesmo conteúdo dos seeders do projeto web:

- **4 usuários:** 1 almoxarife e 3 professores (Programação, Redes, Eletrotécnica)
- **13 livros** em 10 matérias, sendo 5 abaixo do estoque mínimo (1 deles zerado)
- **12 movimentações** (5 entradas e 7 saídas) distribuídas nos últimos 28 dias

Para recomeçar do zero, apague os dados do app (Configurações do Android → Apps → senaistock_mobile → Armazenamento → Limpar dados) ou desinstale e reinstale.

5. Autenticação e Perfis de Acesso

5.1 Fluxo de login

```
Usuário abre o app
↓
Session.restaurar() → há um usuário salvo no aparelho?
↓ não                               ↓ sim
Tela de Login                       Entra direto no app
↓
Preenche e-mail + senha → "Entrar"
↓
Session.login() → AuthRepository.login()
  busca o usuário pelo e-mail + compara o hash da senha
↓
✅ Almojarife → abre na aba Dashboard
✅ Professor → abre na aba Livros (sem Dashboard)
❌ Credenciais erradas → "E-mail ou senha incorretos."
```

5.2 Sessão

- O id do usuário logado é salvo com `shared_preferences`; ao reabrir o app, a sessão é restaurada sem pedir login.
- Sair:** toque no avatar no canto superior direito → **Sair**. O id salvo é apagado e o app volta ao login.

5.3 Perfis e permissões

Recurso	Almojarife	Professor
Dashboard com estatísticas	✅	❌ (abre direto em Livros)
Livros — visualizar	✅ todos	✅ só da sua matéria
Livros — criar / editar / excluir	✅	❌
Movimentações — registrar entrada/saída	✅	❌
Movimentações — ver histórico	✅	❌
Reservas — criar nova	❌	✅
Reservas — dar baixa	✅ (todas)	❌
Reservas — cancelar	✅ (todas)	✅ (só as próprias)
Reservas — visualizar	✅ (todas)	✅ (só as próprias)
Notificações de chegada de livros	❌	✅
Gerenciar usuários	✅	❌

5.4 Como o controle de acesso é feito

No web, o middleware `EnsureRole` bloqueia as rotas. No app não existem rotas digitáveis, então o controle é feito em duas camadas:

- Navegação:** `HomeShell._abasPara()` monta a barra inferior conforme o perfil — o professor simplesmente não tem as abas Dashboard, Estoque e Usuários, e os botões de criar/editar/excluir não

aparecem para ele.

2. **Repositórios:** as regras que dependem de quem está logado também são verificadas na camada de dados — o filtro por matéria do professor (`LivroRepository.listar`), a lista de reservas só com as próprias (`ReservaRepository.listar`) e a permissão de cancelamento (`ReservaRepository.cancelar`).

Barra inferior do Almoxarife

Barra inferior do Professor

Dashboard · Livros · Estoque · Reservas · Usuários

Livros · Reservas · Notificações

5.5 Senhas

As senhas são guardadas como hash **SHA-256** (`PasswordHasher`), nunca em texto puro. O projeto web usa **bcrypt**, que é mais seguro (tem *salt* e custo computacional ajustável); a escolha do SHA-256 aqui é adequada apenas para um app de demonstração com banco local — ver [Limitações](#).

6. Módulos do Sistema

6.1 Dashboard — Almoxarife

`lib/screens/dashboard/dashboard_screen.dart`

- **4 cards de estatísticas:** total de títulos, livros em baixo estoque, entradas hoje, saídas hoje
- **Baixo estoque:** até 10 livros com `saldo_atual ≤ estoque_minimo`, do menor saldo para o maior
- **Movimentações recentes:** as 6 últimas, com badge de entrada/saída e quantidade
- **Reservas pendentes:** as 5 mais recentes aguardando baixa
- **Atualização:** puxe a tela para baixo (*pull to refresh*). No web a atualização é automática a cada 60 s (`wire:poll`).

6.2 Livros

`lib/screens/livros/`

Almoxarife:

- Lista com busca instantânea por título, ISBN ou matéria e filtro por matéria (chips roláveis)
- Botão **Novo livro**; tocar no card abre a edição; o menu no badge de saldo oferece **Editar** e **Excluir**
- Badge de saldo colorido: ● normal | ● abaixo do mínimo | ● zerado
- Exclusão com diálogo de confirmação — bloqueada se o livro tiver movimentações ou reservas
- Formulário: título, ISBN (único), matéria e estoque mínimo. Livro novo começa com saldo 0; na edição, o saldo é exibido mas não pode ser alterado (só muda por movimentações)

Professor:

- Mesma lista, **filtrada automaticamente pela sua matéria** (aviso "Mostrando livros de ...")
- Somente leitura, sem botões de criar/editar/excluir
- Busca por título ou ISBN

6.3 Movimentações (aba "Estoque") — Almoxarife

`lib/screens/movimentacoes/`

Histórico:

- Lista de todas as entradas e saídas, da mais recente para a mais antiga
- Filtros: tipo (Todos / Entrada / Saída), livro e período (seletor de intervalo de datas)
- Botão **Limpar filtros** aparece quando há algum filtro ativo
- Cada item mostra: livro, data/hora, quem registrou, observação, badge do tipo e quantidade

Registrar (botão **Registrar**):

- **Livro:** abre uma lista de todos os livros agrupados por matéria, com campo de busca por título ou matéria e o badge de saldo de cada um (equivale ao combobox Alpine.js do web)
- **Tipo:** cards Entrada (verde) ou Saída (vermelho)
- **Quantidade:** mínimo 1
- **Observação:** obrigatória para saídas, opcional para entradas
- Ao salvar: atualiza o saldo, grava a movimentação e, se for entrada, notifica os professores da matéria; a mensagem de confirmação informa quantos professores foram notificados
- Saída maior que o saldo é recusada com "Estoque insuficiente. Saldo atual: X | Solicitado: Y."

6.4 Reservas

lib/screens/reservas/

Fluxo completo:

```
Professor cria a solicitação (vários livros num único envio)
↓
Reservas ficam com status "Pendente"
↓
Almoxarife vê no Dashboard e na aba Reservas
↓
Almoxarife toca em "Dar baixa" → diálogo de confirmação
↓
Sistema valida o saldo → registra saída automática → status vira "Retirada"
```

Professor — Nova Solicitação (botão **Nova reserva**):

- Campo de turma/motivo (obrigatório)
- Lista com todos os livros da sua matéria: caixa de seleção + quantidade (botões – e +, habilitados só quando o livro está marcado; desmarcar volta a quantidade para 1)
- Badge de saldo em cada livro
- Resumo dinâmico dos livros selecionados na parte de baixo da tela
- Botão **Enviar solicitação (N)** desabilitado até selecionar ao menos um livro
- Um único envio cria todas as reservas numa **transação SQL**

Almoxarife — Gestão de reservas:

- Filtro por status: Todas / Pendentes / Retiradas / Canceladas
- Reservas pendentes têm os botões **Dar baixa** e **Cancelar**, ambos com confirmação
- A quantidade fica em vermelho, com o saldo atual ao lado, quando o estoque não cobre a reserva
- Dar baixa cria automaticamente uma saída no estoque com a observação "Baixa de reserva #N — Professor — Turma"

Professor — Minhas reservas: vê somente as próprias e pode cancelar as que ainda estão pendentes.

6.5 Notificações — Professor

`lib/screens/notificacoes/notificacoes_screen.dart`

- Lista de avisos de chegada de livros da matéria do professor
- Badge vermelho na aba Notificações com a contagem de não lidas
- As notificações são marcadas como lidas automaticamente
- Remover: botão × ou arrastar o card para a esquerda
- Mensagem: *Chegaram 10 exemplar(es) de "Algoritmos e Lógica de Programação" (Programação).*

6.6 Usuários — Almoxarife

`lib/screens/usuarios/`

Lista:

- Avatar com a inicial do nome (âmbar para almoxarife, azul para professor), nome, e-mail, matéria e badge do perfil
- O próprio usuário logado aparece com "(você)" e sem o botão de excluir
- Busca por nome ou e-mail e filtro por perfil (Todos / Almoxarifes / Professores)
- Proteções na exclusão: não exclui a si mesmo, não exclui o único almoxarife e não exclui usuário com registros vinculados (movimentações, reservas ou notificações)

Criar / editar usuário:

- Nome, e-mail (formato validado e único), perfil escolhido em cards visuais
- Campo **Matéria** aparece quando o perfil é Professor, com sugestões das matérias já cadastradas nos livros
- Senha com no mínimo 6 caracteres + confirmação
- Na edição, a senha é opcional (em branco mantém a atual)
- O almoxarife não pode alterar o próprio perfil (evita perder o acesso por engano)

7. Regras de Negócio

Regra	Descrição	Onde é aplicada
RN1	Estoque não pode ficar negativo	<code>EstoqueRepository.registrarSaida()</code> — lança <code>EstoqueInsuficienteException</code>
RN2	Quantidade deve ser maior que zero	<code>EstoqueRepository</code> (<code>QuantidadeInvalidaException</code>) + validação dos formulários
RN3	ISBN deve ser único por livro	Coluna <code>UNIQUE</code> + <code>LivroRepository.criar()</code> / <code>atualizar()</code>
RN4	Toda movimentação registra usuário e data/hora	<code>EstoqueRepository</code> grava <code>user_id</code> e <code>data_hora</code>
RN5	Entrada/saída são atômicas	<code>db.transaction()</code> em <code>registrarEntrada()</code> e <code>registrarSaida()</code>
RN6	Alerta quando saldo $\leq$ estoque mínimo	<code>Livro.estaBaixoEstoque</code> , Dashboard, <code>SaldoBadge</code>
RN7	Reserva só pode ser criada por professor	<code>ReservasScreen</code> — o botão Nova reserva só existe para o professor
RN8	Professor vê apenas livros da sua matéria	<code>LivroRepository.listar()</code> — filtro SQL por <code>usuario.materia</code>
RN9	Entrada notifica os professores da matéria	<code>RegistrarMovimentacaoScreen</code> → <code>NotificacaoRepository.notificarProfessoresDaMateria()</code>
RN10	Dar baixa valida o saldo antes de registrar a saída	<code>ReservaRepository.darBaixa()</code> — <code>Livro.temSaldoSuficiente()</code>

Regras complementares (também presentes no web):

Regra	Onde é aplicada
Livro com movimentações não pode ser excluído	<code>LivroRepository.excluir()</code>
Usuário não pode excluir a si mesmo	<code>UsuarioRepository.excluir()</code>
Sempre deve existir ao menos um almoxarife	<code>UsuarioRepository.excluir()</code>
E-mail de usuário é único	Coluna <code>UNIQUE</code> + <code>UsuarioRepository.criar()</code> / <code>atualizar()</code>
Reserva já retirada ou cancelada não pode ser processada de novo	<code>ReservaRepository.darBaixa()</code> / <code>cancelar()</code>
Almoxarife não pode alterar o próprio perfil	<code>UsuarioFormScreen</code>

8. Camada de Dados (no lugar da API RESTful)

O app **não possui API HTTP**: as telas chamam diretamente os **repositórios**, que fazem as consultas no SQLite e aplicam as regras de negócio. Os repositórios cumprem o papel que, no web, é dos controllers da API, do `EstoqueService` e dos métodos dos componentes Livewire.

Repositório	Método	Equivalente no projeto web	O que faz
AuthRepository	login(email, senha)	LoginController::login / AuthService::login	Valida credenciais e retorna o usuário
	buscarPorId(id)	Auth::user()	Restaura o usuário da sessão
	listar(usuario, busca, materia)	Livros/Index · GET /api/livros	Lista com busca, filtro e regra RN8
LivroRepository	buscarTodosParaFiltro()	lista do combobox em Registrar	Todos os livros, por matéria e título
	listarMaterias()	Livro::distinct('materia')	Matérias para filtros e sugestões
	criar(...)	Livros/Criar · POST /api/livros	Cadastra com ISBN único
	atualizar(...)	Livros/Editar	Edita (ISBN continua único)
	excluir(id)	Livros/Index::excluir	Exclui, se não houver movimentações
EstoqueRepository	registrarEntrada(...)	EstoqueService::registrarEntrada · POST /api/stock/entries	Soma ao saldo + grava movimentação
	registrarSaida(...)	EstoqueService::registrarSaida · POST /api/stock/exits	Valida saldo, subtrai + grava movimentação
	listarBaixoEstoque(minimo?)	EstoqueService::listarBaixoEstoque · GET /api/stock/low	Livros com saldo ≤ mínimo
MovimentacaoRepository	listar(tipo, livro, inicio, fim)	Movimentacoes/Index · GET /api/movimentacoes	Histórico com filtros
ReservaRepository	listar(usuario, status)	Reservas/Index	Todas (almoxarife) ou as próprias (professor)
	criar(professor, itens, obs)	Reservas/Nova::salvar	Várias reservas numa transação
	darBaixa(id, almoxarife)	Reservas/Index::darBaixa	Valida saldo, gera saída, marca "retirada"
	cancelar(id, usuario)	Reservas/Index::cancelar	Marca "cancelada"
NotificacaoRepository	notificarProfessoresDaMateria(livro, qtd)	NovaEntradaLivro + notify()	Cria uma notificação por professor da matéria

Repositório	Método	Equivalente no projeto web	O que faz
	<code>listarEMarcarLidas(usuarioId)</code>	<code>Notificacoes/Index::mount</code>	Lista e marca como lidas
	<code>contarNaoLidas(usuarioId)</code>	contador da sidebar	Badge da aba Notificações
	<code>excluir(id)</code>	<code>Notificacoes/Index::excluir</code>	Remove uma notificação
<code>UsuarioRepository</code>	<code>listar(busca, perfil)</code>	<code>Usuarios/Index</code>	Lista com busca e filtro
	<code>criar(...)</code> / <code>atualizar(...)</code>	<code>Usuarios/Criar</code> / <code>Usuarios/Editar</code>	Cadastro e edição com e-mail único
	<code>excluir(id, logadoId)</code>	<code>Usuarios/Index::excluir</code>	Exclusão com as proteções da seção 6.6
<code>DashboardRepository</code>	<code>carregar()</code>	<code>Dashboard::render</code>	Estatísticas e listas do Dashboard

Tratamento de erros

Em vez de códigos HTTP (422, 403...), os repositórios lançam **exceções de domínio**

(`lib/core/app_exceptions.dart`), que as telas capturam e mostram ao usuário numa mensagem vermelha:

Exceção	Quando	Equivalente HTTP no web
<code>EstoqueInsuficienteException</code>	Saída ou baixa maior que o saldo (RN1)	422
<code>QuantidadeInvalidaException</code>	Quantidade ≤ 0 (RN2)	422
<code>RegraNegocioException</code>	ISBN/e-mail duplicado, exclusão bloqueada, reserva já processada, falta de permissão	422 / 403

9. Serviço de Estoque

`EstoqueRepository` — `lib/repositories/estoque_repository.dart`

Centraliza a lógica de estoque, isolando-a das telas — exatamente o papel do `EstoqueService.php`.

Método	Parâmetros	O que faz
<code>registrarEntrada()</code>	<code>Livro, int quantidade, Usuario, String observacao</code>	Soma ao saldo + grava a movimentação numa transação
<code>registrarSaida()</code>	<code>Livro, int quantidade, Usuario, String observacao</code>	Valida o saldo (RN1) + subtrai + grava numa transação
<code>listarBaixoEstoque()</code>	<code>int? minimo</code>	Livros com <code>saldo_atual ≤ estoque_minimo</code> (ou <code>≤ minimo</code>)

Fluxo interno de uma saída:

```
registrarSaida() chamado
↓
quantidade <= 0 → QuantidadeInvalidaException (RN2)
↓
!livro.temSaldoSuficiente(quantidade) → EstoqueInsuficienteException (RN1)
↓
db.transaction() {
    UPDATE livros SET saldo_atual = saldo_atual - quantidade
    INSERT INTO movimentacoes (livro_id, user_id, tipo='saida', quantidade, observacao, data_hora=agora)
}
```

Se qualquer passo da transação falhar, o SQLite desfaz tudo (*rollback*): o saldo e o histórico nunca ficam dessincronizados (RN5).

A baixa de reserva (`ReservaRepository.darBaixa`) reaproveita este mesmo `registrarSaida()`, então as regras de saída valem também para as reservas.

10. Notificações

O app usa uma **tabela de notificações no banco local**, no mesmo modelo do canal `database` do Laravel. Não há notificações *push* do sistema Android.

```
Almoxarife registra uma ENTRADA (RegistrarMovimentacaoScreen)
↓
EstoqueRepository.registrarEntrada() concluído
↓
NotificacaoRepository.notificarProfessoresDaMateria(livro, quantidade)
  SELECT * FROM users WHERE perfil = 'coordenador' AND materia = livro.materia
↓
Para cada professor encontrado:
  INSERT INTO notificacoes (id = UUID, user_id, livro_id, livro_titulo,
                           materia, quantidade, mensagem, created_at)
↓
Mensagem ao almoxarife: "Entrada registrada com sucesso! N professor(es) de <matéria> notificado(s)."
```

Interface do professor:

- Badge vermelho na aba **Notificações** com a quantidade de não lidas
- As não lidas são marcadas como lidas automaticamente (`read_at`)
- Cada notificação pode ser removida individualmente

Como o banco é local, a notificação só aparece para o professor que fizer login **no mesmo aparelho** em que a entrada foi registrada — ver [Limitações](#).

11. Credenciais de Demonstração

Perfil	E-mail	Senha	Matéria
Almoxarife	almoxarife@senai.br	senha123	—
Professor	coordenador@senai.br	senha123	Programação
Professor	prof.redes@senai.br	senha123	Redes
Professor	prof.eleto@senai.br	senha123	Eletrotécnica

A tela de login já vem preenchida com o almoxarife para agilizar demonstrações.

Roteiro sugerido para demonstrar o fluxo completo num único aparelho:

1. Entre como **almoxarife**, registre uma **entrada** de "Algoritmos e Lógica de Programação" e saia.
2. Entre como **professor** (`coordenador@senai.br`): veja a notificação, abra **Reservas** → **Nova reserva**, peça alguns livros e saia.
3. Entre de novo como **almoxarife**: a reserva aparece no Dashboard; em **Reservas**, toque em **Dar baixa** e confira a saída automática em **Estoque**.

12. Como Executar

12.1 Pré-requisitos

Ferramenta	Versão usada	Local de instalação usado
Git	2.5x	—
Flutter SDK	3.47.5 (stable)	C:\src\flutter
JDK	Temurin 21	C:\src\jdk21
Android SDK (cmdline-tools)	ver seção 2.3	C:\Android\sdk
Virtualização	Hyper-V / Windows Hypervisor Platform ativo	(necessário para o emulador)

Não é necessário instalar o Android Studio.

12.2 Montando o ambiente numa máquina nova (Windows)

```
# 1. Flutter (pelo git – o .zip oficial apresentou falha de inicialização nesta máquina)
git clone -b stable --depth 1 https://github.com/flutter/flutter.git C:\src\flutter

# 2. JDK 21 portátil: baixar o .zip do Eclipse Temurin 21 e extrair em C:\src\jdk21
#   https://adoptium.net/temurin/releases/?version=21

# 3. Android command-line tools: baixar em https://developer.android.com/studio#command-tools
#   e extrair de forma que exista C:\Android\sdk\cmdline-tools\latest\bin\android.exe

# 4. Variáveis de ambiente do usuário
[Environment]::SetEnvironmentVariable("JAVA_HOME", "C:\src\jdk21", "User")
[Environment]::SetEnvironmentVariable("ANDROID_HOME", "C:\Android\sdk", "User")
#   e adicionar ao Path: C:\src\flutter\bin; C:\src\jdk21\bin; C:\Android\sdk\platform-tools;
#                       C:\Android\sdk\cmdline-tools\latest\bin; C:\Android\sdk\emulator
#   (feche e reabra o terminal depois)

# 5. Pacotes do Android SDK
android --no-metrics sdk install platform-tools "platforms/android-36" "build-tools/36.0.0" emulator "system-images/android-36/google_apis_playstore/x86_64"

# 6. NDK 28.2.13676358 (ver 12.6 se travar)
android --no-metrics sdk install "ndk/28.2.13676358"

# 7. Emulador
android --no-metrics emulator create medium_phone

# 8. Conferência
flutter doctor
```

12.3 Rodando o app

```
cd "Projeto Mobile"
flutter pub get

# Liga o emulador (ou conecte um celular Android com "Depuração USB" ativa)
android --no-metrics emulator start medium_phone

flutter run
```

Com o `flutter run` aberto no terminal: `r` = *hot reload*, `R` = *hot restart*, `q` = encerrar.

O **primeiro build** demora vários minutos (cerca de 8 a 18 minutos nesta máquina), pois o Gradle baixa e compila as dependências Android. Os builds seguintes levam segundos.

12.4 Gerando o APK para instalar em celulares

```
flutter build apk --release
# Arquivo gerado: build\app\outputs\flutter-apk\app-release.apk
```

O APK pode ser copiado para o celular e instalado (é preciso permitir "instalar apps de fontes desconhecidas"). Atualmente ele é assinado com a chave de depuração — ver [Limitações](#).

12.5 Comandos úteis

Comando	Para quê
<code>flutter devices</code>	Lista emuladores e celulares conectados
<code>flutter analyze</code>	Análise estática do código
<code>flutter test</code>	Roda os testes automatizados
<code>flutter clean</code>	Apaga os arquivos de build (use se o build ficar inconsistente)
<code>adb uninstall</code> <code>com.senai.senaistock.senaistock_mobile</code>	Desinstala o app (zera o banco local)

12.6 Problemas conhecidos e soluções

Problema	Causa	Solução
Primeiro build parado em "Running Gradle task 'assembleDebug'" por muito tempo, sem uso de CPU	O downloader interno do Gradle trava ao baixar a distribuição nesta rede	Baixar com <code>curl</code> e colocar em <code>%USERPROFILE%\.gradle\wrapper\dists\gradle-9.3.1-all\9ot9r568e8zfvd4mn8rbu1j0\gradle-9.3.1-all.zip</code>
<code>Package ndk not found</code> / falha ao chamar <code>sdkmanager.bat</code>	O NDK não estava instalado e a instalação automática (ferramenta antiga) falha	Instalar o NDK antes (passo 6). Se travar, baixar <code>android-ndk-r28c-windows.zip</code> com <code>curl</code> e extrair com <code>tar -xf</code> em <code>C:\Android\sdk\ndk\28.2.13676358</code>
<code>Expand-Archive</code> muito lento	O PowerShell extrai arquivo por arquivo, e o antivírus escaneia cada um	Usar <code>tar -xf arquivo.zip</code>
"Acesso negado" logo após extrair algo	O antivírus segura arquivos recém-criados por alguns segundos	Tentar de novo após alguns segundos
<code>Warning: SDK processing. This version only understands SDK XML versions up to 3...</code>	A CLI do Android é mais nova que o plugin Gradle	Apenas um aviso; pode ser ignorado

13. Testes e Qualidade

13.1 Análise estática

`flutter analyze` usa as regras do pacote `flutter_lints` mais duas regras do projeto (`analysis_options.yaml`):

- `prefer_const_constructors` — componentes imutáveis declarados como `const` (melhor desempenho)
- `prefer_single_quotes` — padrão de aspas simples no código Dart

Resultado atual: **No issues found!**

13.2 Testes automatizados

`test/widget_test.dart` — **6 testes, todos passando** (`flutter test`):

Teste	Regra verificada
Livro está em baixo estoque quando saldo $\leq$ mínimo	RN6
Saldo suficiente só quando cobre a quantidade pedida	RN1
Reserva indica quando o saldo não cobre a quantidade pedida	Destaque em vermelho (apoio à RN10)
Só reservas pendentes podem receber baixa	Ciclo de vida da reserva
Perfil <code>coordenador</code> é exibido como "Professor"	Perfis
Hash de senha confere apenas com a senha correta	Autenticação

13.3 Validação manual

O app foi executado no emulador Android 16 (API 36): login como almoxarife, Dashboard com os dados de demonstração e navegação até o formulário de registro de movimentação, sem erros no log do dispositivo.

14. Diferenças em Relação ao Projeto Web

Aspecto	Projeto web	Projeto mobile
Navegação	Sidebar lateral	Barra de navegação inferior (abas)
Atualização do Dashboard	Automática a cada 60 s (<code>wire:poll</code>)	Manual: puxar a tela para baixo
Paginação	15–20 itens por página	Lista contínua com rolagem
Seleção de livro em "Registrar"	Combobox Alpine.js	Lista agrupada por matéria com busca, em painel inferior
Período no histórico	Dois campos de data	Seletor de intervalo de datas
Remover notificação	Botão	Botão × ou arrastar para a esquerda
Badge de reservas pendentes no menu do almoxarife	✓	⚠ Não implementado (pendentes aparecem no Dashboard e na aba Reservas)
Dados	Compartilhados por todos no servidor	Locais em cada aparelho
API RESTful	✓ Sanctum	Não se aplica

15. Limitações Conhecidas e Próximos Passos

Legenda:  Implementado |  Restrição conhecida |  Futuro

Item	Detalhe
 Dados locais por aparelho	Cada celular tem seu próprio banco. Uma reserva feita no celular do professor não chega ao celular do almoxarife. Hoje o fluxo completo funciona com os perfis usando o mesmo aparelho .
 Senhas com SHA-256 sem <i>salt</i>	Suficiente para demonstração; em produção, usar bcrypt/argon2 ou autenticação no servidor.
 Notificações marcadas como lidas cedo demais	Todas as abas são montadas no login, então a tela de Notificações marca os avisos como lidos assim que o professor entra no app, e não só quando ele abre a aba. Correção prevista: montar as abas sob demanda.
 Listas não se atualizam sozinhas entre abas	Após registrar uma movimentação, o Dashboard e a lista de Livros mostram o saldo novo após puxar a tela para baixo.
 Sem badge de reservas pendentes para o almoxarife	Existe no web; aqui as pendentes aparecem no Dashboard e na aba Reservas.
 Nome do app no celular aparece como "senaistock_mobile"	Ajustar <code>android:label</code> no <code>AndroidManifest.xml</code> para "SenaiStock".
 APK de release assinado com chave de depuração	Necessário criar uma chave própria antes de distribuir.
 Sincronização com o servidor Laravel	Estender a API do projeto web (reservas, notificações, usuários, dashboard) e trocar os repositórios locais por chamadas HTTP, mantendo as telas.
 Notificações <i>push</i>	Avisar o professor mesmo com o app fechado (ex.: Firebase Cloud Messaging), dependente da sincronização.
 Versão iOS	O código Flutter é o mesmo; falta gerar e testar o projeto iOS (requer macOS).
 Leitura de ISBN pela câmera	Agilizar o cadastro e as movimentações escaneando o código de barras do livro.
 Testes de widget e de integração	Cobrir as telas e os repositórios com banco SQLite em memória.

16. Log de Desenvolvimento

Fase 1 — Análise do projeto de referência (02/09/2026)

- Leitura completa do projeto web (`Projeto/`): models, migrations, seeders, `EstoqueService` , controllers da API, componentes Livewire, middleware de perfis e documentação
- Levantamento dos módulos, perfis, regras RN1–RN10 e fluxos de tela a reproduzir

Fase 2 — Decisão de arquitetura

- **Tentativa inicial:** estender a API Laravel com endpoints de Dashboard, Reservas, Notificações e Usuários para o app consumir
- **Revertida** a pedido da equipe, para **não alterar o projeto original** — o repositório voltou exatamente ao estado anterior
- **Decisão final:** app independente, com banco SQLite local e as regras de negócio reimplementadas em Dart

Fase 3 — Modelos e banco de dados

- Enums `PerfilUsuario` , `TipoMovimentacao` , `StatusReserva` (mesmos valores do web)
- Models `Usuario` , `Livro` , `Movimentacao` , `Reserva` , `NotificacaoLivro`
- `AppDatabase` : schema das 5 tabelas com chaves estrangeiras e dados de demonstração idênticos aos seeders

Fase 4 — Regras de negócio (repositórios)

- `EstoqueRepository` espelhando o `EstoqueService` (RN1, RN2, RN4, RN5, RN6)
- Repositórios de livros, movimentações, reservas (com baixa e transação), notificações, usuários e dashboard
- Exceções de domínio no lugar dos códigos HTTP

Fase 5 — Telas e navegação

- Login com sessão persistente (`Session` + `shared_preferences`)
- `HomeShell` com barra inferior por perfil (equivalente à sidebar dinâmica)
- Telas de Dashboard, Livros, Movimentações, Reservas, Notificações e Usuários, com formulários, filtros, badges e diálogos de confirmação

Fase 6 — Ambiente e primeira execução (02/09/2026)

- Instalação do Flutter: o `.zip` oficial apresentou falha de inicialização → instalado via `git clone`
- Android SDK instalado pela nova **Android CLI** (`android sdk install`), que substitui o `sdkmanager`
- Primeiro build travou baixando o Gradle → distribuição baixada com `curl` e colocada no cache
- Build falhou por falta do **NDK 28.2.13676358** (e a instalação automática pelo `sdkmanager.bat` quebrava) → NDK baixado com `curl` e extraído com `tar`
- App executado no emulador Android 16: login e Dashboard validados

Fase 7 — Reconstrução e melhorias (01/10/2026)

- A máquina do laboratório foi resetada e os arquivos do app e do ambiente se perderam
- Projeto reconstruído integralmente; ambiente reinstalado com **JDK Temurin 21 portátil** (o Android Studio também havia sido removido)
- Downloads de Gradle e NDK feitos direto por `curl` desde o início, sem os travamentos da primeira vez
- **Melhorias:**
 - `heroTag` únicos nos botões flutuantes (evita erro ao abrir formulários com várias abas montadas)
 - Seletor de livro em "Registrar" agrupado por matéria e com busca
 - Filtro por período no histórico de movimentações
 - Resumo dinâmico e quantidade habilitada só para livros marcados na Nova Solicitação
 - Validações de e-mail e de confirmação de senha no próprio formulário; bloqueio de alteração do próprio perfil
 - Testes automatizados das regras de negócio (6 testes) e `flutter analyze` sem avisos
- Primeiro build concluído em ~8,5 minutos; app instalado e validado no emulador
- Documentação técnica (este arquivo) e `README.md`

Legenda:  Implementado |  Restrição conhecida |  Futuro